

Python Lecture 5 시험 공부 정리본

파일 입출력 / Lecture 5 단독 보충 자료

사용 방법

이전 Lecture 1-4 자료와 별도로 공부하는 마지막 범위 정리본이다. PDF 17쪽과 Lecture5-1.py 전체를 참고했다. 각 파트는 개념 → 수업 예제 → 체크 → 마무리 4문제 → 힌트로 구성한다. 총 16문제이며 정답은 별도 파일이다.

예제의 input.txt·dummy.txt·output.txt는 첨부되지 않았다. 강의에 나온 이름과 문자열 외에 A/B 파일은 동작을 명확히 보여주기 위한 학습용 가정이다. 마지막 개행 유무는 문제마다 명시했다. encoding, for 반복, write 반환값, closed는 보충이며 출제 여부를 단정하지 않는다.

학습 지도

01 파일 모드·현재 위치 → 02 읽기 메소드 비교 → 03 반복 읽기·개행·EOF → 04 쓰기·달기·with

핵심 한 줄

read는 전체 문자열, readline은 한 줄 문자열, readlines는 줄 문자열들의 리스트. w는 기존 내용 삭제, a는 끝에 추가. EOF는 빈 문자열이지 빈 줄이 아니다.

PART 01 파일 열기·모드·현재 위치

자료: PDF p.2-4

핵심 개념

파일 입출력은 열기 → 읽기/쓰기 → 닫기 순서이다. open()은 파일 객체를 반환하며 변수명 infile/f는 사용자가 정하는 이름이다. 파일명과 모드는 문자열로 전달한다.

```
infile = open("input.txt", "r", encoding="utf-8")
# 파일 객체 infile로 작업
infile.close()
```

encoding은 자료 원본에는 없지만 한글을 안정적으로 처리하기 위한 보충이다. 실제 파일의 인코딩과 맞아야 한다. 상대 경로 input.txt는 실행 중인 현재 작업 폴더 기준이며 .py 파일 위치와 항상 같지는 않다.

모드 비교

- r: 읽기만, 기존 내용 유지, 시작 위치 처음, 없으면 FileNotFoundError.
- r+: 읽기·쓰기, 기존 내용 유지, 시작 위치 처음, 없으면 FileNotFoundError.
- w: 쓰기만, 열 때 기존 내용을 모두 지움, 없으면 생성.
- w+: 읽기·쓰기, 열 때 기존 내용을 모두 지움, 없으면 생성.
- a: 쓰기만, 기존 내용 유지, 쓰기는 끝에 추가, 없으면 생성.
- a+: 읽기·쓰기, 기존 내용 유지, 열면 끝 위치, 쓰기는 끝에 추가, 없으면 생성.

+는 읽기와 쓰기 모두 가능하게 한다. w는 write를 실행하지 않아도 여는 순간 내용이 잘린다. a에서 추가되는 것은 전달한 문자열뿐이며 줄바꿈이 자동으로 생기지는 않는다.

seek와 위치

읽으면 현재 위치가 앞으로 이동한다. f.seek(0)은 처음으로 돌아간다. r+는 기존 내용 중 현재 위치부터 덮어쓰므로 파일 전체를 자동 교체하는 모드가 아니다. a 계열의 쓰기는 seek로 위치를 옮겨도 끝에 추가된다. seek는 모드 자체를 바꾸는 기능이 아니라 위치를 옮기는 기능이다.

```
with open("sample.txt", "w+", encoding="utf-8") as f:
    f.write("ABC")
    f.seek(0)
    print(f.read()) # ABC
```

쓰기 뒤 위치는 끝이므로 처음부터 읽으려면 seek(0)을 사용한다. 텍스트 파일에서 임의의 바이트 위치를 계산하는 문제는 여기서 다루지 않는다.

암기 체크

□ r은 파일 필요 □ w는 열 때 삭제 □ a는 끝에 추가 □ +는 읽기·쓰기 □ seek는 위치 변경

마무리 연습문제

문제 01-1

기존 파일에 ABC가 있다. w로 열고 아무것도 쓰지 않은 채 닫았다. 남은 내용은?

풀이 메모: _____

문제 01-2

없는 파일을 r, w, a로 열 때의 차이를 설명하라.

풀이 메모: _____

문제 01-3

w+로 ABC를 쓴 직후 read()를 호출하면? 처음부터 읽도록 한 줄을 추가하라.

풀이 메모: _____

문제 01-4

기존 내용 ABC에 a 모드로 X를 썼다. 최종 내용과 줄바꿈 유무는?

풀이 메모: _____

막힐 때만 보는 힌트

- 01-1: write 호출 전에도 w의 효과가 있다.
- 01-2: 읽을 기존 파일이 필요한 모드를 찾자.
- 01-3: 파일의 현재 위치는 어디인가?
- 01-4: 추가 모드는 기존 내용을 유지한다.

PART 02 read·readline·readlines 비교

자료: PDF p.6-11

세 메소드의 반환형

- f.read(): 현재 위치부터 남은 전체 내용을 문자열(str) 하나로 반환.
- f.readline(): 현재 위치부터 한 줄을 문자열(str)로 반환. 보통 끝의 개행 문자 \n도 포함.
- f.readlines(): 남은 줄들을 문자열 원소의 리스트(list)로 반환.

자료의 read 설명 중 개행을 보존하지 않는다는 표현은 주의해야 한다. read() 결과에는 개행이 남아 있다. print는 개행을 실제 줄바꿈으로 보여주고 repr는 \n 표기를 보여준다. 일반 텍스트 모드에서 줄 끝 표기가 \n으로 정규화 될 수 있다는 점과 개행 자체가 제거된다는 것은 다르다.

동일한 파일로 비교

다음은 학습용 파일 가정이다: 내용은 정확히 A\nB, 마지막 B 뒤에는 개행이 없다. 각 예제는 파일을 새로 열어 실행한다.

```
with open("sample.txt", "r", encoding="utf-8") as f:
    text = f.read()
    print(repr(text))      # 'A\nB'
    print(type(text))     # <class 'str'>
    items = []
    items.append(text)
    print(items)          # ['A\nB']
```

read 결과를 append하면 전체 문자열이 원소 하나로 들어가므로 리스트 길이는 1이다. 줄별 원소가 되는 것은 아니다.

```
with open("sample.txt", "r", encoding="utf-8") as f:
    print(repr(f.readline())) # 'A\n'
    print(repr(f.readline())) # 'B'
    print(repr(f.readline())) # ''
with open("sample.txt", "r", encoding="utf-8") as f:
    print(f.readlines())     # ['A\n', 'B']
```

readline은 다음 줄로 이동한다. 마지막 줄에 원래 개행이 없다면 개행을 붙이지 않는다. EOF(파일 끝)에서 readline()/read()는 '', readlines()는 []를 반환한다. 먼저 한 줄 읽고 readlines()하면 나머지 줄만 얻는다.

강의 dummy.txt 연결

자료는 The Big Short, The Cathedral, Sherlock Holmes, Matilda, Le Petit Prince의 5줄을 예로 든다. read는 5 줄 전체 문자열 하나, readlines는 줄 문자열 5개이다. 자료 출력 예시에서 마지막 Le Petit Prince에는 개행이 없다.

암기 체크

☐ read/readline은 str ☐ readlines는 list ☐ 개행 포함 ☐ 현재 위치부터 읽음

마무리 연습문제

문제 02-1

파일 내용이 정확히 A\WnB이다. 새로 열어 read()한 값의 repr 표현과 자료형은?

풀이 메모: _____

문제 02-2

같은 파일을 새로 열고 readline()을 세 번 호출했다. 각 반환값의 repr 표현은?

풀이 메모: _____

문제 02-3

같은 파일을 새로 열어 readline() 한 번 후 readlines()를 호출했다. 결과는?

풀이 메모: _____

문제 02-4

items=[]에 read() 결과를 append했다. 파일이 5줄일 때 len(items)는? 줄별 리스트를 얻는 메소드는?

풀이 메모: _____

막힐 때만 보는 힌트

- 02-1: repr는 개행을 눈에 보이는 표기로 바꾼다.
- 02-2: 마지막 줄에 개행이 없다는 전제를 보자.
- 02-3: 파일 위치가 첫 줄 뒤로 이동했다.
- 02-4: append는 한 번 실행됐다.

PART 03 반복 읽기·개행·EOF·rstrip 함정

자료: PDF p.12-14 / Lecture5-1.py 읽기 두 블록

필기 코드의 실행 흐름

```
infile = open("input.txt", "r")
line = infile.readline()
while line != "":
    print(line)
    line = infile.readline()
infile.close()
```

① 먼저 한 줄 읽음 ② 빈 문자열이 아니면 출력 ③ 다음 줄 읽음 ④ 조건으로 돌아감 ⑤ EOF의 ''를 만나 종료 ⑥ 닫음. 갱신문이 없으면 같은 line을 반복 출력한다. 처음 읽는 문장이 없으면 line을 정의하지 않고 사용할 수 있다.

print(line)과 print(line, end='')

line 자체에 개행이 있는데 print도 기본 end='\n'을 붙이므로 줄 사이에 빈 줄이 생길 수 있다. 원래 파일의 개행을 그대로 쓰려면 print(line, end='')를 사용한다.

학습용 파일 내용이 정확히 홍길동\n김철수\n이면 첫 코드가 만드는 출력 문자열의 repr는 '홍길동\n\n김철수\n\n'이다. end=''인 코드는 '홍길동\n김철수\n'이다. 마지막 줄의 개행 유무에 따라 마지막 출력의 줄 끝은 달라진다.

EOF와 빈 줄은 다르다

EOF는 ''. 파일 안의 빈 줄은 '\n'이다. while line != ''는 빈 줄을 만나도 계속 읽는다.

```
# 안전한 형태: 원문으로 EOF 판정 후 출력용으로 가공
with open("input.txt", "r", encoding="utf-8") as infile:
    line = infile.readline()
    while line != "":
        print(line.rstrip("\n"))
        line = infile.readline()
```

rstrip()는 뒤쪽 공백·탭·개행 등을 제거한다. rstrip('\n')는 끝의 개행 문자만 제거한다(일반 텍스트 모드 가정). 문자열은 불변이므로 새 문자열이 반환된다.

강의의 rstrip 종료 조건 주의

자료 p.14처럼 line=infile.readline().rstrip()를 먼저 실행하고 while line!=''로 검사하면 빈 줄이나 공백만 있는 줄이 ''로 변해서 EOF로 오인되고 조기 종료된다. 파일이 정확히 A\n\nB\n이면 A만 출력하고 B는 읽지 못한다. 위 안전한 코드처럼 원문으로 EOF를 판정한 뒤 출력할 때 가공하자.

보충: 간단한 반복

```
with open("input.txt", "r", encoding="utf-8") as infile:
    for line in infile:
        print(line, end="")
```

파일 객체를 직접 반복하면 줄 단위로 읽으며 EOF에서 자동으로 반복을 마친다.

암기 체크

□ EOF '' / 빈 줄 'wn' □ 다음 줄 갱신 □ 이중 개행 □ 가공 전 EOF 검사

마무리 연습문제

문제 03-1

정확히 AwnBwn인 파일을 print(line)으로 줄별 출력할 때 전체 출력 문자열의 repr는?

풀이 메모: _____

문제 03-2

readline()이 빈 줄에서 반환하는 값과 EOF에서 반환하는 값을 비교하라.

풀이 메모: _____

문제 03-3

정확히 AwnwnBwn인 파일을 p.14의 rstrip 후 while 조건 코드로 읽으면 무엇이 출력되는가? 이유는?

풀이 메모: _____

문제 03-4

직접 작성: input.txt를 끝까지 읽어 원래 개행을 유지해 출력하라. with와 while을 사용하라.

풀이 메모: _____

막힐 때만 보는 힌트

- 03-1: 개행이 두 곳에서 생긴다.
- 03-2: 줄 내용이 없다는 것과 더 읽을 것이 없다는 것은 다르다.
- 03-3: rstrip은 빈 줄의 개행도 지운다.
- 03-4: 첫 읽기·조건·출력·갱신의 순서를 쓰자.

PART 04 write·close·with·필기 전체 추적

자료: PDF p.5,15-16 / Lecture5-1.py 쓰기·닫기 블록

파일에 쓰기

```
with open("output.txt", "w", encoding="utf-8") as outfile:
    outfile.write("김영희\n")
```

output.txt가 없으면 생성, 있으면 내용 삭제 후 새로 작성. write는 문자열을 받아 파일에 기록하며 개행을 자동 추가하지 않는다. write(123)는 TypeError이므로 write(str(123))처럼 변환한다. 텍스트 write의 반환값은 쓴 문자 수이며 바이트 수와 다르다. '김영희\n'은 문자 4개이다(보충).

필기는 쓰기 모드 파일에도 변수명 infile을 사용한다. 이것은 문법 오류가 아니다. 읽기/쓰기 가능 여부를 결정하는 것은 변수명이 아니라 open의 모드이다.

close와 with

close()는 파일을 닫고 버퍼에 남은 쓰기 내용을 처리한다. 닫힌 파일에 read/write를 시도하면 ValueError. 파일 객체 변수 자체가 사라지는 것은 아니며 f.closed는 True이다(보충).

with open(...) as f:의 들여쓰기 블록을 벗어나면 파일이 자동으로 닫힌다. 블록 안에서 예외가 발생해도 닫히도록 정리되지만 예외 자체를 없애 주지는 않는다. 없는 파일을 r로 여는 오류도 해결해 주지 않는다.

```
with open("test.txt", "w", encoding="utf-8") as f:
    f.write("공부")
# 이 위치에서는 파일이 닫힌 상태
```

with 본문에 주석만 적으면 실행 문장이 없어 IndentationError가 난다. 설명용 틀을 실제 실행하려면 pass 또는 실제 작업을 넣는다.

```
with open("test.txt", "w") as f:
    pass
```

이 코드는 파일을 닫아도 빈 파일을 만든다. 기존 내용은 w로 열 때 지워진다.

필기 전체를 실행하면

Lecture5-1.py는 input.txt를 첫 블록에서 읽고 닫은 뒤, 두 번째 블록에서 다시 열어 같은 내용을 다시 읽는다. 즉 화면에 내용이 두 차례 나온다(첫 블록은 이중 개행 가능). 이어 output.txt에 김영희와 개행을 기록하고, test.txt는 w로 열기만 하고 닫아 빈 파일이 된다.

input.txt가 없으면 첫 open에서 오류가 발생해 뒤쪽 쓰기 코드도 실행되지 않는다. 필기 맨 아래 #with... 예시는 모두 주석이므로 실행되지 않는다. 테스트는 실습용 파일로 하고 중요한 기존 파일에 w를 쓰지 말자.

암기 체크

☐ write는 str ☐ 자동 개행 없음 ☐ with 자동 닫기 ☐ 예외 제거는 아님 ☐ 첫 오류에서 후속 중단

마무리 연습문제

문제 04-1

f.write("김영희\n")의 텍스트 모드 반환값은? f.write(123)은 어떻게 수정하는가?

풀이 메모: _____

문제 04-2

with로 파일을 읽으면 파일이 없어도 오류 없이 실행되는가? 설명하라.

풀이 메모: _____

문제 04-3

필기를 처음부터 실행할 때 input.txt가 존재한다. 파일 읽기는 몇 차례이며 test.txt에는 무엇이 남는가?

풀이 메모: _____

문제 04-4

직접 작성: output.txt의 기존 내용을 유지하면서 새 줄 문자열 "김철수\n"를 끝에 추가하고 자동으로 닫히게 하라.
기존 내용은 이미 개행으로 끝난다고 가정한다.

풀이 메모: _____

막힐 때만 보는 힌트

- 04-1: 텍스트 문자 수와 저장 바이트 수를 구별하자.
- 04-2: 자동 닫기와 예외 처리는 다른 기능이다.
- 04-3: open("input.txt", "r")이 두 번 나온다.
- 04-4: 기존 내용 유지에는 w가 아니라 다른 모드가 필요하다.

시험 직전 최종 체크

- r/r+는 기존 파일이 필요하다. □ w/w+는 여는 순간 내용을 지운다.
- a/a+의 쓰기는 끝에 추가한다. □ +는 읽기·쓰기 모두 가능하다.
- seek(0)은 처음으로 이동한다. □ 읽기는 현재 위치부터 진행한다.
- read/readline은 str, readlines는 list. □ read는 개행을 제거하지 않는다.
- EOF는 '', 빈 줄은 'wn'. □ 마지막 줄에 개행이 없을 수도 있다.
- print(line)은 추가 줄바꿈, end=''는 추가 줄바꿈 없음.
- rstrip 후 EOF 판정은 빈 줄에서 조기 종료될 수 있다.
- write는 문자열을 받으며 개행은 직접 쓴다.
- with는 자동으로 닫지만 예외를 없애지 않는다.
- 필기의 두 읽기 블록은 같은 파일을 각각 처음부터 읽는다.

공부 순서

첫 회독: 모드 6개와 읽기 메소드 3개를 비교해서 설명한다. 둘째 회독: 빈 줄·EOF·개행을 repr로 직접 확인한다. 마지막: 16문제를 풀고 틀린 문제는 현재 파일 위치와 문자열 값을 한 줄씩 추적한다.